

Week 4 — MCP server over the Person database

John Michael Talbo · jmjmlang Repo: jmjmlang/ai-agent-dev-week4-mcp-server

What I set out to do

Week 4 is the payoff for everything earlier. Week 1 set up MCP clients pointed at someone else's servers. Week 3 built a real database with auth and CRUD. This week joins them: an MCP server, written by me, that exposes Week 3's Person table as tools an agent can call.

The brief in my head: "give Claude or VS Code Copilot a tool called `create_person` that actually writes to the Postgres I provisioned last week, and make sure two different keys can't see each other's rows."

What got built

A single Next.js API route at `/api/mcp` running the Streamable HTTP MCP transport from `@modelcontextprotocol/sdk`. Five tools:

- `list_people(limit?)`
- `get_person(id)`
- `create_person(name, email?, role?, notes?)`
- `update_person(id, ...fields)`
- `delete_person(id)`

Plus a tiny `npm run issue-key` script that creates an API key for an existing user. Plus client configs (`.vscode/mcp.json` and a Claude Desktop equivalent using `mcp-remote`).

How auth works

MCP servers don't have a sign-in flow. So the auth model is API keys, the same pattern as the GitHub or Linear MCP servers I configured in Week 1.

I added one new table — `ApiKey` — to the Week 3 schema. It has `keyHash` (sha256 of the plaintext), `userId`, a name, and a `revokedAt` for kill switching. The plaintext key is only shown at creation time and looks like `wkmcp_<64 hex chars>`. The `wkmcp_` prefix makes it grep-able in logs.

Every request to `/api/mcp` runs through `authenticate()` first. It pulls the bearer token, hashes it, looks up the row, checks `revokedAt`, and returns `{ userId }` on success. That `userId` is then passed into a freshly-built `McpServer` whose tools all include `ownerId: userId` in their Prisma where clauses. Same security boundary as the Week 3 web app — the server *cannot* return another account's people, even if a tool got called with a guessed id.

`updateMany` and `deleteMany` are used instead of `update` / `delete` for the same reason: they let you put `ownerId` in the where clause without needing a separate "find first" round-trip, and a 0-row result naturally tells you "not yours / not found."

Schema strategy

Week 3 already runs migrations against this database. Two projects fighting over migration history is a recipe for pain. So Week 4 uses `prisma db push` instead of `migrate dev`. The schema mirrors Week 3 exactly (so Prisma doesn't try to drop tables it doesn't know about) and only adds the `ApiKey` model. `db push` saw that everything else already existed and just created the new table.

Production-grade approach would be to add this table back to the Week 3 project as a proper migration, then both projects share one source of truth. For a workshop this is fine and keeps the projects loosely coupled.

Stack notes

- `@modelcontextprotocol/sdk 1.2.9` ships a `WebStandardStreamable HTTPServerTransport` whose `handleRequest(req: Request)` returns a `Response`. That maps perfectly onto a Next.js route handler — no Express shim needed.
- **Stateless mode** (`sessionIdGenerator: undefined`, `enableJsonResponse: true`) means each request gets a fresh transport and server. No session map to maintain, no sticky sessions on Vercel, no resume support — fine for tool calls. If I ever needed long-running streams, I'd revisit.
- **Prisma 7 + Neon adapter** — same setup as Week 3 (the Neon serverless driver, not the standard Postgres TCP one). Vercel's serverless runtime doesn't keep persistent connections; the Neon driver uses HTTP/WebSocket fetches, which is the right primitive here.

Things that went sideways

1. **Prisma Studio hijacked the terminal.** I tried piping `prisma studio --browser none` together with a node one-liner to find the user id. The studio command never exits — it runs a server. Killed it. Used a plain node script to query the user directly.
2. `npm pkg set` **choked on** `=` **in PowerShell** when I tried to chain multiple script entries. Edited `package.json` directly.
3. **Build was almost incident-free this time** — by week 4 the Prisma 7 + Tailwind v4 + Next 16 quirks from previous weeks were already routine.

What this unlocks

The whole point of this stack is what happens next. With this server live, I can:

- In VS Code Copilot Chat: ask "list the people I added last week" and watch it call `list_people` and read back rows from Postgres.
- In Claude Desktop (via `mcp-remote`): say "add Jane to my contacts as CTO" and have it call `create_person`.
- Compose with the other Week 1 servers (rolldice, calendar booking, GitHub) — the agent picks whichever tool fits the request.

That's the MCP architecture loop closed: I am no longer just *using* someone else's MCP servers. I'm shipping one.

Deploy

Vercel auto-detected Next.js. `postinstall` runs `prisma generate`. The build script is plain `next build` — no migrations to run from this project, since Week 3 owns the schema timeline. One env var: `DATABASE_URL`. The deployed `/api/mcp` URL is what the Claude and VS Code configs point at.

Open items

- The deployed Vercel URL — recorded after deploy.
- A revocation UI on the Week 3 app would be nice (right now you'd have to edit the `revokedAt` column directly via Prisma Studio). Not in scope for this week.
- Rate limiting. The Neon free tier is the only thing keeping this from being abused if a key leaked. A real production version would put a per-key rate limit in front of the route.

Wrapping the four weeks

- Week 1 — set up MCP clients pointed at hosted servers. Learned the config shape (`mcp.json`, transports, headers).
- Week 2 — built a static personal CV. Got fluent with Next.js 15 + React 19 server components.
- Week 3 — built a real CRUD app with Prisma + Neon + Auth.js. The first week with a database under it.
- Week 4 — built an MCP server in front of that database, so an AI agent can use it as a tool. The architecture diagram now reads *Agent* ↔ *MCP server* ↔ *Postgres*, all four layers shipped by me.

The single biggest mindset change across the four weeks is the one flagged in week 1: I'm not coding for the next compiler error any more. I'm coding for whatever model is going to read this code and decide which tool to call. Tool names matter. Descriptions matter. Error messages matter — they're the agent's only way to recover. Server-side authorization matters because the agent will absolutely try to call a tool with the wrong id at some point.

That's the workshop. Time to actually use the thing for something.